AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

...Algorytm listy dwukierunkowej z zastosowaniem GitHub...

Autor:
Kamil Kwiczal

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Wymagania funkcjonalne i projektowe 3

1.2. Wymagania narzędziowe 3

2. Analiza problemu **5**

2.1. Zastosowanie algorytmu 5

2.2. Sposób działania – przykład ręczny 5

2.3. Narzędzia wykorzystane w projekcie 6

3. Projektowanie **7**

4. Implementacja **8**

5. Wnioski **9**

Literatura **10**

Spis rysunków **10**

Spis tabel **11**

Spis listingów **12**

1. Ogólne określenie wymagań

Głównym celem niniejszego projektu jest zaprojektowanie i zaimplementowanie struktury danych w postaci listy dwukierunkowej, alokowanej dynamicznie na stercie. Aplikacja zostanie napisana w języku C++ i wykorzysta sprawdzone wzorce projektowe: wzorzec fabryki (ang. *Factory*) do kontrolowanego tworzenia nowych węzłów oraz wzorzec iteratora (ang. *Iterator*) do sprawnego i bezpiecznego poruszania się po elementach kolekcji.

1.1. Wymagania funkcjonalne i projektowe

Zgodnie z założeniami projektowymi, klasa implementująca listę dwukierunkową musi udostępniać następujące operacje i metody:

- dodawanie elementu na początek, na koniec oraz pod wskazany indeks,
- usuwanie elementu z początku, z końca oraz z podanego indeksu,
- wyświetlanie zawartości całej listy oraz wyświetlanie w odwrotnej kolejności,
- przechodzenie do następnego i poprzedniego elementu (wykorzystując Iterator),
- całkowite czyszczenie listy z dealokacją pamięci, co zapobiega wyciekom.

Każda klasa wchodząca w skład projektu musi znajdować się w oddzielnym pliku nagłówkowym (`.h`) oraz źródłowym (`.cpp`). Poprawność działania wszystkich napisanych metod zostanie rygorystycznie zweryfikowana w głównej funkcji programu (`main`).

1.2. Wymagania narzędziowe

Poza samą strukturą danych, projekt ma na celu zapoznanie z nowoczesnymi narzędziami programistycznymi. Kod źródłowy będzie od samego początku wersjonowany za pomocą systemu Git i przechowywany w zdalnym repozytorium GitHub. W ramach prac zaplanowano przetestowanie konkretnych scenariuszy z użyciem systemu kontroli wersji, takich jak: tworzenie rewizji (minimum 5 commitów), cofanie zmian (revert), usuwanie historii, praca na wielu lokalizacjach (klonowanie repozytorium do innego folderu) oraz przywracanie usuniętych plików, które psułyby kompilację.

Dodatkowo kod źródłowy C++ zostanie wyposażony w komentarze zgodne ze standardem `Doxygen`, co pozwoli na automatyczne wygenerowanie dokumentacji kodu w formacie PDF.

2. Analiza problemu

Lista dwukierunkowa to dynamiczna struktura danych, w której każdy element (węzeł) przechowuje nie tylko same dane, ale również dwa wskaźniki: jeden wskazujący na element poprzedni (`prev`), a drugi na element następny (`next`). Zapewnia to możliwość iterowania po liście w obu kierunkach, co rozwiązuje ograniczenia standardowej listy jednokierunkowej.

2.1. Zastosowanie algorytmu

Listy dwukierunkowe znajdują szerokie zastosowanie w praktyce programistycznej, szczególnie tam, gdzie wymagane jest szybkie poruszanie się w przód i w tył bez konieczności ciągłego przeszukiwania kolekcji od nowa. Klasycznymi przykładami są:

- **Przeglądarki internetowe:** implementacja historii przeglądania (strzałki „Wstecz" i „Dalej").
- **Odtwarzacze multimedialne:** zarządzanie playlistami (przełączanie utworów na poprzedni/następny).
- **Aplikacje biurowe i graficzne:** mechanizmy historii akcji (często implementowane jako systemy Undo/Redo).

2.2. Sposób działania – przykład ręczny

Aby zwizualizować działanie programu, przeanalizujmy ręcznie proces wpinania i usuwania elementów. Przyjmujemy, że nasza lista przechowuje typ całkowity (`int`), a główna klasa zarządza wskaźnikami `Head` (początek) oraz `Tail` (koniec).

Krok 1: Inicjalizacja. Lista jest pusta. Wskaźniki `Head` i `Tail` mają przypisaną wartość `nullptr`.

Krok 2: Dodanie liczby 10 na początek. Tworzony jest nowy węzeł [10]. Ponieważ to jedyny element, zarówno `Head`, jak i `Tail` wskazują na [10]. Jego własne wskaźniki `prev` i `next` wynoszą `nullptr`.

```
Head -> [10] <- Tail
```

Krok 3: Dodanie liczby 20 na koniec. Wzorzec fabryki generuje węzeł [20]. Wskaźnik `next` dotychczasowego elementu [10] ustawiany jest na nowy węzeł [20]. Wskaźnik `prev` węzła [20] ustawiany jest na [10]. `Tail` przesuwa się i wskazuje na [20].

```
Head -> [10] <-> [20] <- Tail
```

Krok 4: Wstawienie liczby 15 pod indeks 1. Zostaje odnaleziony element aktualnie pod indeksem 1 (czyli [20]). Nowy węzeł [15] aktualizuje wskaźniki sąsiadów: `next` węzła [10] zaczyna wskazywać na [15], `prev` węzła [15] to [10], `next` węzła [15] to [20], a `prev` węzła [20] od teraz to [15].

```
Head -> [10] <-> [15] <-> [20] <- Tail
```

Krok 5: Usunięcie z końca elementu (20). Wskaźnik `Tail` cofa się na poprzedni element, czyli [15]. Wskaźnik `next` elementu [15] jest zerowany (`nullptr`). Pamięć zarezerwowana wcześniej na stercie dla węzła [20] zostaje całkowicie zwolniona (np. używając operatora `delete`).

```
Head -> [10] <-> [15] <- Tail
```

2.3. Narzędzia wykorzystane w projekcie

Realizacja projektu w środowisku programistycznym wspierana jest przez zestaw profesjonalnych narzędzi:

- **Git oraz GitHub:** Git to rozproszony system kontroli wersji pozwalający na bezpieczne tworzenie kolejnych paczek zmian (commitów). Umożliwia m.in. swobodne pobieranie modyfikacji i powrót do starszych wersji. GitHub to usługa hostingowa, która zapewnia zdalne przechowywanie repozytorium.
- **Doxygen:** Zautomatyzowane narzędzie do dokumentowania oprogramowania. Doxygen parsuje kod napisany w języku C++ i potrafi wygenerować spójną i czytelną dokumentację w formacie PDF.

3. Projektowanie

4. Implementacja

5. Wnioski

Spis rysunków

Spis tabel

Spis listingów